Exercise 6 — k-NN, Naive Bayes and Support Vector Machines

Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

Set 20 November 2026 · discussed at the start of lesson 7, Friday 27 November 2026

Contents

What this exercise is for . . . 1
The factory . . . 1
Part 1 — Predict, before you fit anything (20 marks) . . . 2
Part 2 — Measure (25 marks) . . . 2
Part 3 — Explain the inversion (30 marks) . . . 3
Part 4 — Mark yourself (25 marks) . . . 3
Marking . . . 3
Getting help . . . 4

What this exercise is for

Lesson 6 compared three families on one dataset and they finished 0.933, 0.944, 0.947 — close enough that the choice between them looked like a detail.

That was an accident of the data. This exercise gives you **two problems from the same factory**, and the ranking of the three families is different at each one. Neither ordering can be guessed from the names of the algorithms; both can be predicted from twenty minutes spent looking at the data.

That is the skill being assessed. Not tuning — **choosing**.

Keep a single notebook, `exercise06.ipynb`, that runs top to bottom in the course container. Written answers go in markdown cells beside the evidence for them.

Nothing is handed in: lesson 7 opens by discussing this exercise, and at the exam one of the ten is drawn for you to talk through your own notebook.

The factory

A compounding plant buys polymer pellets by the sack, melts them in a twin-screw extruder, and sells the compounded product against a melt-flow specification. Two things go wrong, they are measured by two completely different instruments, and they are wrong in two completely different shapes.

`batch_data.py` ships with this exercise and needs no network.

```python
from batch_data import load_spectra, load_extrusion

X_a, y_a = load_spectra()      # goods-inwards screening
X_b, y_b = load_extrusion()    # the extruder's energy monitor
```

Station A — the near-infrared spectrometer. Every incoming sack is scanned across 80 wavelength channels. The failure is a grade mix-up: a sack of recycled material on a pallet of virgin. The lab reference that produces the labels is expensive, so there are only 200 of them, and the reference method has an error rate of its own.

Station B — the extruder energy monitor. Every production run records throughput and heater power. The failure is a batch that misses the melt-flow specification.

Read the module docstring. It describes the physics of both stations honestly and at length, and it is not padding: everything you need to *predict* the results is in it, which is the point of Part 1.

`batch_data.py` also carries `TRUE_*` constants recording how the labels were generated. **Do not read them until Part 4.**

Part 1 — Predict, before you fit anything (20 marks)

1. Load both datasets and report, for each: the number of rows, the number of columns, the positive rate, and the majority-class baseline.
2. Look at the data. Plot whatever helps. Do **not** fit a model yet.
3. Then, in writing, **predict the ranking** of these five at each station: logistic regression, Gaussian Naive Bayes, k-NN, a linear support vector machine, and an SVM with a radial basis function kernel.

   Give a reason per station, in terms of lesson 6: the shape of the boundary, the number of dimensions against the number of rows, and whether the features are independent given the class.

What is being marked: the reasoning, not the ranking. A confident wrong prediction that engages with the right properties scores well; a hedged list that commits to nothing does not. **Write the prediction down before you run anything** — the rest of the exercise is much less instructive if you do not.

Part 2 — Measure (25 marks)

4. Cross-validate all five models at both stations. Report accuracy with its spread across folds, using the machinery of lesson 5 — not a single split.
5. Present the two stations side by side, with the majority baseline on each.
6. State plainly where your Part 1 predictions were wrong.

What is being marked: that the comparison is honest — same folds, same preprocessing inside the pipeline, scaling applied where the method needs it — and that question 6 is answered rather than quietly skipped.

One warning, worth more than it looks: **one of these two stations has 200 rows.** Think about what lesson 5 said about the width of an error bar before you declare any winner there.

Part 3 — Explain the inversion (30 marks)

The ranking is not the same at the two stations. Explain why, in a paragraph of six to ten sentences per station.

Support each explanation with a **measurement**, not an assertion. Suggestions, though you may find better ones:

- for the claim that the features are or are not independent given the class: compute the within-class correlations and quote them — remembering that a correlation well away from zero proves dependence, while one near zero rules out only the straight-line kind (the handout's section 4.3);
- for a claim about the curse of dimensionality: sweep `k` and show what happens, or reduce the number of channels and show the effect;
- for a claim that no single feature carries the signal: fit a model on the single best feature and compare against the full set.

What is being marked: this part carries the most marks in the exercise, and the measurements are what separates an explanation from a plausible story. Lesson 6's handout makes several claims of exactly this kind and backs every one with a number; do the same.

Part 4 — Mark yourself (25 marks)

Now open `batch_data.py` and read the `TRUE_*` constants.

7. At station A: how many of the 80 channels actually carry any signal, and how large is the effect on the strongest one, in units of detector noise? Compare that against what your models needed.
8. At station A there is a **dead channel** — one photodiode that contributes nothing. Did anything in your analysis reveal it? If not, what would have?
9. At station B: what rule generates the label? Having seen it, explain in two or three sentences why the two linear models scored where they did.
10. Two things set a ceiling below 1.0, both in the measurement process rather than the model: errors in the reference labels, and readings that overlap between the classes, so that some sacks or batches look alike whatever their true state. Which of the two can you compute at each station from the `TRUE_*` constants, what does it give, and how close did your best model get?

What is being marked: question 9. Being able to look at a generating rule and say *why* a family of models could or could not represent it is the whole content of lesson 6.

Marking

Part	Marks
1 — Predict before fitting	20
2 — Measure	25
3 — Explain the inversion	30
4 — Mark yourself	25
Total	**100**

Marks are lost for:

- reporting a single-split score instead of a cross-validated one (**−10**);
- fitting a scaler outside the pipeline (**−10**);
- declaring a winner at station A without acknowledging the width of the error bar there (**−5**);
- a notebook that does not run top to bottom in the container (**−10**).

Marks are **not** lost for a wrong prediction in Part 1, nor for a model that performs poorly, provided the report is honest about both. A wrong prediction you then investigated is worth more here than a cautious one.

Getting help

Email fabio.antonini.1969@gmail.com. There is no lesson between the day this is set and the morning it is due, so the inbox is the only channel. If you are stuck on Part 3, notebook 2 of this lesson does exactly this kind of measurement — it finds the within-class correlations near zero, shows the readings are dependent all the same, and then measures why Naive Bayes works anyway.